

Executable Archaeology: Reanimating the Logic Theorist from its IPL-V Source

Jeff Shrager, Ph.D.
Bennu Climate, Inc.
jshrager@gmail.com

Abstract—The Logic Theorist (LT), created by Allen Newell, J. C. Shaw, and Herbert Simon in 1955–1956, is widely regarded as the first artificial intelligence program. While the original conceptual model was described in 1956, it underwent several iterations as the underlying Information Processing Language (IPL) evolved. Here I describe the construction of a new IPL-V interpreter, written in Common Lisp, and the faithful reanimation of the Logic Theorist from code transcribed directly from Stefferud’s 1963 RAND technical report. Stefferud’s version represents a pedagogical re-coding of the original heuristic logic into the standardized IPL-V. The reanimated LT successfully proves 16 of 23 attempted theorems from Chapter 2 of *Principia Mathematica*, results that are historically consistent with the original system’s behavior within its search limits. To the author’s knowledge, this is the first successful execution of the original Logic Theorist code in over half a century. A postscript describes David Moews’s complementary reanimation of the never-machine-executed 1956 IPL-I version, which came to light after this work was first reported, and which sharpens the question of what executable archaeology contributes to the history of AI.

Index Terms—Logic Theorist, IPL-V, IPL-I, artificial intelligence history, software archaeology, A. Newell, H. A. Simon, J. C. Shaw, list processing, digital preservation

I. INTRODUCTION

In the summer of 1956, a small group of researchers gathered at Dartmouth College for a workshop on what John McCarthy had dubbed “artificial intelligence.” Among them were Allen Newell and Herbert Simon, who arrived with something none of the other attendees had: a program—albeit one not yet running on any machine—that could discover proofs of mathematical theorems.¹

The Logic Theorist (LT) [1], [2] would ultimately prove 38 of the first 52 theorems in Chapter 2 of Whitehead and Russell’s *Principia Mathematica* [5], in some cases finding proofs more elegant than those produced by the human authors. For Theorem 2.85, LT discovered a shorter, more direct proof than the one published in *Principia*. Simon showed the new proof to Bertrand Russell himself, who “responded with delight” [14, p. 167]. Edward Feigenbaum, then an undergraduate taking a graduate course from Simon, recalls the moment the project’s significance was first announced. As he later told Pamela McCorduck: “It was just after Christmas vacation—January

1956—when Herb Simon came into the classroom and said, ‘Over Christmas Allen Newell and I invented a thinking machine.’ And we all looked blank” [14, p. 138].

Beyond reproducing *Principia* proofs, the Logic Theorist introduced heuristic search strategies—means–ends analysis and subgoal decomposition—derived from protocol analyses of human problem solving. These ideas later became foundational in both artificial intelligence and cognitive architectures such as the General Problem Solver (GPS) [18], Soar, ACT, and EPAM [19]. LT thus sits at the very beginning of the fusion of AI and cognitive psychology.

LT was implemented in the Information Processing Language (IPL), a language created by the same team—Newell, Shaw, and Simon—specifically for building cognitive models [6]. IPL introduced list manipulation, dynamic memory allocation, symbols as first-class objects, recursion, and higher-order functions, and was an explicit, acknowledged ancestor of Lisp [7] and of other early list-processing systems such as Weizenbaum’s SLIP. Yet while Lisp has thrived for nearly seven decades, IPL has been extinct since the mid-1960s. Although numerous IPL-V implementations existed in the 1960s on various machines, no maintained implementations appear to be available in modern environments. The only prior modern attempt known to the author is Richman’s unpublished IPL-V interpreter, written in Microsoft BASIC, which he used to run EPAM IV in the early 1990s [20].²

This paper describes the construction of a complete IPL-V interpreter in Common Lisp, and the faithful reanimation of the Logic Theorist from code transcribed directly from Stefferud’s 1963 RAND technical report [11]. More than 99% of the LT code running in this reconstruction is the original code from that report. The reanimated LT proves 16 of 23 attempted theorems from *Principia Mathematica*, results that are historically consistent with the original LT’s known behavior.

This work illustrates a methodological approach that might be called *executable archaeology*: the study of historically significant computational systems through faithful reconstruction and execution of their original code. Early AIs were not merely theoretical proposals but functioning programs whose detailed behavior cannot be fully understood from published descriptions alone. Reanimating such systems provides a new empirical window into the origins of artificial intelligence;

¹It is worth noting that Art Samuel’s checkers-playing program existed a couple of years before LT, and it too was heuristic. But Samuel’s goal was to build a checkers player, whereas Newell, Simon, and Shaw’s explicit goal was to model human cognition across many domains. Samuel’s effort was brilliant in its own right—he also built arguably the first machine learning program, and the first system to learn from self-play.

²Richman wrote to the author (personal communication, Sep 26, 2025, quoted with permission): “I just interpreted IPL-V in Microsoft Basic using the IPL-V manual that I found at the CMU library.”

Section VII takes up exactly what that window shows, and the Postscript (Section X) describes a complementary reanimation, by David Moews, of the program’s never-machine-executed 1956 ancestor, which came to light after this work was first reported.

In prior work, a team including the present author restored Weizenbaum’s original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094 [12]. Although the Logic Theorist and ELIZA addressed very different problems—formal theorem proving versus natural-language conversation—both emerged from the same symbolic, list-processing programming lineage originating in IPL. However, whereas the ELIZA work rested on a stack that emulated vintage hardware and operating systems, the LT reanimation emulates the abstract IPL-V machine.

II. HISTORICAL BACKGROUND

A. The Origins and Versions of the Logic Theorist

The story of the Logic Theorist begins in the early 1950s at the RAND Corporation. Herbert Simon and Allen Newell, collaborating with J. C. (Cliff) Shaw, sought to simulate complex thought by manipulating symbols. However, the history of the program is characterized by a series of shifting specifications across different versions of the Information Processing Language.

The system described in the famous 1956 RAND report (P-868) [1], and in nearly identical form in the IRE Transactions that September [2], was a “paper” version, written in what was called IPL-I or the “Logic Language” (LL). IPL-I was never implemented on any machine: it was a pseudocode for an imagined abstract machine, executed entirely by hand. At the time of the Dartmouth workshop in the summer of 1956, Newell and Simon noted that the program was “essentially specified” and that hand simulations suggested it would prove the 60-odd theorems of Chapter 2 of *Principia Mathematica*. These hand simulations famously employed “the human computer”: Simon’s wife, his children, and graduate students acting as program components [8, Ch. 13].

The first version to run on real hardware was written in IPL-II for the RAND JOHNNIAC [4], [13], producing its first machine proof in August 1956 [4], [8]. The widely cited result that LT proved 38 of the first 52 theorems in *Principia*—including the discovery of a more elegant proof for Theorem 2.85 than the one produced by Whitehead and Russell—stems from the empirical explorations reported in 1957 [3].

The code used in this reanimation, transcribed from Stefferud’s 1963 memorandum, represents a third distinct phase. The program was converted to IPL-V by Fred Tonge and further developed by Stefferud as a pedagogical re-coding of the original heuristic logic into IPL-V, the first version of the language made widely available for public use [11]. Thus, while this reanimation is a faithful execution of original code published by the RAND team, it is a restoration of the 1963 “modernized” version of the 1956 conceptual model. (The Postscript describes the subsequent reanimation of the 1956 version itself.)

TABLE I
EVOLUTION OF THE LOGIC THEORIST AND IPL

Year	Version	Platform	Historical Milestones
1956	IPL-I (LL)	Manual	Conceptual spec; hand-simulated only.
1956–57	IPL-II	JOHNNIAC	First machine proofs; 38 of 52 theorems.
1958	IPL-IV	IBM 704	Internal RAND use; transition to IPL-V.
1963	IPL-V	Various	Stefferud’s pedagogical LT implementation.
2026	IPL-V	Lisp	This reanimation (Stefferud’s source).
2026	IPL-I	Python	Moews: first machine execution (Postscript).

B. IPL: The Invention of Symbol- and List-Processing

To implement LT, Newell, Shaw, and Simon needed a programming language suited to symbolic reasoning. No such language existed, so they created the “Information Processing Language.” As noted, IPL went through several iterations. The earliest versions (IPL-I and IPL-II) powered the 1956 hand simulations and subsequent JOHNNIAC runs; IPL-V, released in the early 1960s [6], became the definitive public standard.

IPL-V runs on an abstract machine organized around *cells*. Each cell has an associated symbol. There is no single distinguished stack; every cell is potentially a stack of symbols that can be pushed and popped. Symbols are central to IPL-V: they are first-class objects, and every cell, data structure, routine, and control element is addressable by a symbol (its name). A set of control cells (H0–H5) governs execution: H0 serves as the parameter and result “communication” cell, H1 serves as both program counter and call stack, H2 is the free list for dynamic memory allocation, and H3 is the cycle counter. Ten working cells (W0–W9) provide scratch storage. Everything in IPL-V—data, routines, even the program itself—is a list of cells, and thus also a list of symbols. Routines are invoked by pushing their symbol (name) onto H1. The IPL machine interprets the operation indicated in that cell, and then continues to whatever is the indicated next cell. Elegantly, because H1 is both program counter and call stack, no special machinery is required to operate the machine aside from basic stack manipulation instructions.

The language’s built-in operations are called J-functions (because their names begin with “J”: J0, J1, J2, and so on). There are approximately 150 of these, implementing operations from basic list manipulation to “generator” (iterator) protocols. In the original IPL-V, many J-functions were themselves written in IPL, although some were implemented at the level of the underlying physical machine.

III. SOURCE MATERIALS AND TRANSCRIPTION

Unlike the ELIZA reanimation, which began with the dramatic discovery of original source code printouts in Weizenbaum’s archives at MIT [12], the source materials for the Logic Theorist were already publicly available. Two documents were indispensable:

- 1) **Stefferud (1963)**, *The Logic Theory Machine: A Model Heuristic Program*, RAND Corporation memorandum RM-3731 [11]. This report contains a complete listing of the LT program in IPL-V—nearly 3,000 lines—along with detailed descriptions of every aspect of LT’s workings, including numerous flow charts.

2) **Newell et al. (1964)**, *Information Processing Language-V Manual*, 2nd edition [6]. The definitive reference for IPL-V, including the semantics of all J-functions, cell structure conventions, and the generator protocol, as well as numerous student exercises and sample programs.

Both documents proved to be exceptionally well-written. There were, however, occasional ambiguities—places where the specification assumed familiarity with conventions of 1960s computing (such as BCD character encoding) that are no longer common knowledge.

The LT code was transcribed from Stefferud’s report into a Google spreadsheet by the author and Anthony Hay. Subsequently, Rupert Lane identified several transcription errors using his “gridlock” tool [16], software designed to accurately extract tabular code from PDF documents. Despite multiple people putting multiple sets of eyes on the code, a surprising number of small transcription errors persisted through numerous rounds of review.³

The transcribed code was converted into an S-expression format (`.lispv` files) suitable for loading by the Lisp-based interpreter. Conveniently, Lisp comment characters work in these files, allowing corrections and annotations to be documented inline.

IV. A NEW IPL-V INTERPRETER IN LISP

I implemented a new IPL-V interpreter to execute the Logic Theorist listing. The interpreter implements the symbolic data structures and execution model described in the IPL-V manual and supports the primitives required by the program. Although the earliest versions of the Logic Theorist ran on the RAND JOHNNIAC, by the time the Stefferud listing, used here, was published (1963) IPL-V had already been implemented on multiple machines. Accordingly, this reconstruction executes the program within a machine-independent IPL-V interpreter rather than attempting to reproduce a specific JOHNNIAC hardware environment. The full interpreter and transcription of the Logic Theorist code are available in the repository associated with this paper.

I chose to emulate the IPL-V abstract machine, rather than emulating the original hardware environment (as was done for ELIZA on the IBM 7094 and CTSS [12]), for several reasons. First, as mentioned just above, the code I am using is IPL-V, and so is not JOHNNIAC-specific. Additionally, no thorough JOHNNIAC emulator exists, and in any case we do not have the JOHNNIAC IPL code. There does exist an IBM 7094 emulator that includes an IPL-V interpreter running under a variant of the IBSYS batch operating system [10], and this emulator includes demo scripts for IPL-V. However, this IPL-V interpreter is a binary with no source code and no documentation. I attempted to run LT on this platform but it failed early in execution with no useful information about the cause. Because there is no source code, this failure was essentially undebuggable.

More fundamentally, however, one of my central goals in undertaking this project was not merely to get LT to run, but

to *elucidate* IPL-V—to develop a deep understanding of the language, the machine model, and the infrastructure of the period. Writing my own interpreter was essential to that goal. IPL-V is defined as an abstract machine specification, not as a program for a specific computer; it was implemented on many different machines through the mid-1960s before being supplanted by Lisp.

My interpreter (`iplv.lisp`) implements the complete IPL-V abstract machine as described in the 1964 manual: the cell table, the control cells H0–H5, the working cells W0–W9, and the push-down mechanisms. Lists are stored as linked chains of cells, following the manual’s specification. It is worth noting that the push-down (stack) mechanism is itself implemented as linked lists within the IPL-V system. Push and pop operations, which appear to be hardware primitives of the abstract machine, are internally built on the same list mechanism that underlies everything else in IPL-V, including data structures, routines, and IPL programs themselves; like Lisp, IPL-V is lists all the way down.

The core of my interpreter is a function, `ipl-eval`, that fetches and executes instructions by executing the list named in the program counter (H1). My IPL evaluator is re-entrant—a capability trivially implementable in Lisp—supporting the recursive calls that IPL-V requires. The J-functions are implemented as Lisp functions. While the interpreter implements the full IPL-V machine, only the subset of J-functions required by LT (and a few test programs) is currently implemented, amounting to several dozen of the approximately 150 defined in the manual.

A complete run of LT attempting proofs of 23 theorems is available at [22]. This takes approximately 574,000 IPL machine cycles and creates over 576,000 symbols⁴. It finishes in a few seconds on a typical modern laptop. On the JOHNNIAC at RAND, in the mid 1950s, this likely would have required hours.

V. DEBUGGING AGAINST THE ARCHIVAL RECORD

The most difficult phase of debugging began once the interpreter was stable enough that LT ran without crashing: the program would prove trivial theorems but fail on simple ones, fall into runaway recursions, or fail to recognize a completed proof—anomalies whose causes were far from their symptoms. This experience had an uncanny resonance with the original developers’ accounts. In an interview conducted by Pamela McCorduck for her pioneering history of AI, Cliff Shaw described the most daunting phase of troubleshooting as occurring late in development, when the system was beyond crashes and produced behavior that appeared superficially reasonable, leaving the developers in a quandary as to whether the results were valid consequences of the heuristic logic or actual bugs, in a memory that had become a churning and impenetrable web of interconnected lists [9]. Sixty years later, with the same program, I faced the identical challenge. In this latter phase I used large language models (Google’s Gemini and Anthropic’s Claude) as debugging assistants, performing the grinding work of reading thousands of lines of execution

³I probably shouldn’t have been surprised by this given the number of typos that have persisted through numerous rounds of review of this paper!

⁴Yeah, I was surprised too. It’s really just a coincidence!

traces and tracking machine state across tens of thousands of cycles; a fuller account of that collaboration is beyond the scope of this paper.

One episode, however, bears directly on the methodology at issue here. The final substantive bug was localized to J74, the list-copying function, yet my Lisp implementation appeared to match the manual’s prose description; the specification itself seemed to support the buggy behavior. Resolution came not from documentation but from an artifact: “Simon’s J’s,” a set of original IPL-V punched cards from 1962 preserved by the Computer History Museum [15], representing the production implementation of the J-functions at the moment IPL-V was being standardized. Comparing the 1962 card-deck implementation of J74 with mine confirmed a hypothesized discrepancy, and a one-line patch made LT run completely correctly. The manual’s prose, my careful reading of it, and inference from my code had all been insufficient; the original 1962 source was the ground truth.

VI. WHAT THE LOGIC THEORIST PROVES

The Logic Theorist attempts to prove theorems in propositional logic using the five axioms of *Principia Mathematica*, Chapter 2. The notation used by the original authors is given in Table II.

TABLE II
LOGICAL NOTATION USED BY THE LOGIC THEORIST

Symbol	Meaning
I	Implication ($\rightarrow$)
V	Disjunction ($\vee$)
*	Conjunction ($\wedge$)
-	Negation ($\neg$)
=	Biconditional ($\leftrightarrow$)
. = .	Definitional equality

LT employs several proof methods:

- **Substitution**—substitute variables in an axiom or previously proved theorem to match the goal.
- **Detachment** (modus ponens)—from $A \rightarrow B$ and A , conclude B .
- **Forward chaining**—apply substitution and detachment working forward from known theorems.
- **Backward chaining**—work backward from the goal, seeking premises that would yield it.
- **Sublevel replacement**—replace a subexpression using a known equivalence.

Each proof attempt is bounded by a settable effort limit in terms of IPL machine cycles. (In all below examples this limit is set to 20,000 cycles.) This is a soft cap: it prevents the system from *beginning* a new subproblem exploration once exceeded, but does not interrupt a search in progress.

Table III shows the axioms and definitions that are initially provided to LT for the runs described in the present paper. Importantly, as LT proves a given theorem, that theorem is treated as an axiom and can be used to support subsequent proofs. Thus, LT can be said, in a sense, to learn!

Table IV shows the results of my reanimated LT on the 23 theorems attempted: 16 are proved, and the remaining 7 are

TABLE III
AXIOMS AND DEFINITIONS

Name	Axiom/Def
1.01	$((PIQ) \rightarrow . (-PVQ))$
2.33	$((PVQVR) \rightarrow . ((PVQ) VR))$
3.01	$((P*Q) \rightarrow . -(- (PV-Q))$
4.01	$((P=Q) \rightarrow . ((PIQ) * (QIP)))$
1.2	$((AVA) IA)$
1.3	$((BI (AVB))$
1.4	$((AVB) I (BVA))$
1.5	$((AV (BVC)) I (BV (AVC)))$
1.6	$((BIC) I ((AVB) I (AVC)))$

not—results that are historically consistent with the original LT’s known behavior, given an effort limit of 20,000 IPL machine cycles.

TABLE IV
THEOREMS ATTEMPTED

Name	Formula	Result
2.01	$(PI-P) I-P$	Proved
2.02	$QI (PIQ)$	Proved
2.04	$(PI (QIR)) I (QI (PIR))$	Proved
2.05	$(QIR) I ((PIQ) I (PIR))$	Proved
2.06	$(PIQ) I ((QIR) I (PIR))$	Proved
2.07	$PI (PVP)$	Proved
2.08	PIP	Proved
2.10	$-PVP$	Proved
2.11	$PV-P$	Proved
2.12	$PI--P$	Proved
2.13	$PV---P$	Proved
2.14	$--PIP$	Not proved
2.15	$(-PIQ) I (-QIP)$	Not proved
2.20	$PI (PVQ)$	Proved
2.21	$-PI (PIQ)$	Not proved
2.24	$PI (-PVQ)$	Proved
3.13	$(- (P*Q)) I (-PV-Q)$	Not proved
3.14	$(-PV-Q) I (- (P*Q))$	Not proved
3.24	$- (P*-P)$	Not proved
4.13	$P==P$	Not proved
4.20	$P=P$	Proved
4.24	$P= (P*P)$	Proved
4.25	$P= (PVP)$	Proved

A representative proof trace is shown below. Here LT proves Theorem 2.01, $(PI-P) I-P$:

2.01 (PI-P) I-P		
PROOF FOUND.		
GIVEN	*1.2	(AVA) IA
SUBSTITUTION	.0	(PI-P) IP
SUBLEVEL REPL	2.01	(PI-P) I-P
Q.E.D.		
EFFORT	LIMIT 20000	ACTUAL 5579
SUBPROBLEMS	LIMIT 50	ACTUAL 1
SUBSTITUTIONS	LIMIT 50	ACTUAL 2

The proof begins from Axiom *1.2, $(AVA) IA$, substitutes $-P$ for A to obtain $(-PV-P) I-P$, then applies sublevel replacement (using the fact that P implies PVP) to transform the antecedent, yielding the desired theorem. The Appendix depicts a more complex proof (4.25), and the original output, including both of these proofs, is at [22].

VII. WHAT EXECUTABLE ARCHAEOLOGY CONTRIBUTES

It is reasonable to ask what is learned by running this old code that could not be learned by reading it, or by reading the abundant secondary literature on the Logic Theorist. The present case—taken together with the complementary reanimation described in the Postscript—suggests several answers.

It converts historical claims into checkable ones. The 1956 report claimed, on the basis of hand simulation, that the program would prove the theorems of *Principia* Chapter 2; the 1957 paper reported the famous 38-of-52 result for the JOHN-NIAC version. Until now, such claims rested on testimony and on archival output listings. Reanimation allows them to be tested directly: the reconstructed LT proves 16 of 23 attempted theorems under a fixed effort limit, a result consistent with the original reports. Moreover, the reconstruction is now an experimental instrument: effort limits, axiom orderings, and theorem sequences can be varied, and the consequences observed on the genuine historical artifact rather than on a modern paraphrase of it.

It shows that some semantics survive only in artifacts, not in documentation. The J74 episode (Section V) is a small but pointed demonstration: the IPL-V manual’s prose description of a core primitive was consistent with an incorrect implementation, and the discrepancy could be resolved only by consulting the original 1962 punched-card source preserved by the Computer History Museum. For the history of computing this is a methodological moral: specifications and manuals, however well written, systematically under-determine the systems they describe, and executable reconstruction is one of the few ways to discover where.

It corrects and annotates the published record. Faithful reconstruction imposes a discipline that reading does not: every typo, ambiguity, and inconsistency in the source materials must be found, repaired, justified, and recorded, because the machine will not silently forgive them. The transcription hazards encountered in working from the 1963 report—and, as the Postscript describes, the outright errors in the 1956 printed listing—are now documented in the project repositories. These repairs are the “archaeology” in executable archaeology.

It makes the gap between cognitive theory and implementation measurable. This contribution emerges most clearly from the juxtaposition of the present work with Moews’s reanimation of the 1956 IPL-I version (Postscript). The 1956 program—roughly 400 lines—is something close to a direct expression of Newell and Simon’s hypothesized cognitive operators for human theorem proving: a high-level specification of a theory of cognition. It could afford to be that, because its “hardware” was human—intelligent, flexible, and able to silently fill in whatever the specification left unsaid. The Stefferud listing—nearly 3,000 lines—is what the same design became when Shaw and his successors had to make a real, literal-minded machine do the whole thing end-to-end. Executing both versions converts a familiar qualitative observation about early AI—that getting from idea to running program was most of the work—into something that can be measured, localized, and studied line by line. Most strikingly, as Moews discovered, the 1956 program contains no machin-

ery at all to assemble or print the proofs it finds: that labor was silently delegated to the human simulators. The omission is itself historical evidence—it marks precisely a boundary of what Newell and Simon considered the *theory* (finding the proof) versus mere clerical work (writing it down).

VIII. RELATED WORK

The reanimation of historical software systems has gained momentum in recent years. Most directly related is the ELIZA reanimation by Lane et al. [12], in which a team including the present author restored Weizenbaum’s original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094. That project reconstructed the complete original hardware and operating system stack; the present work instead emulates the abstract machine specification in a modern language.

Reimplementing the Logic Theorist’s *algorithms* in a modern language is a relatively straightforward exercise—essentially an introductory AI course project, and one that has surely been done many times. A 1968 paper, for example, describes an implementation of LT in LISP 1.5 [17]. The present work is a fundamentally different undertaking: rather than rewriting LT’s logic in a modern language, it constructs a faithful emulator of the IPL-V abstract machine and runs the original code, transcribed directly from a 1963 technical report. The distinction is significant: a Lisp reimplementation demonstrates that LT’s algorithms can be expressed in Lisp (which is unsurprising), while the present work demonstrates that the original IPL-V code, as written by Newell, Simon, and Shaw’s team, functions as described, and, moreover, enables us to perform experiments on the real, working system of machine and program.

IX. NEXT STEPS

The IPL-V interpreter developed for this project is not specific to the Logic Theorist. It implements the general IPL-V abstract machine and can, in principle, run any IPL-V program. I am in the process of building a corpus of early IPL-V AI programs. The most significant targets are the General Problem Solver (GPS) [18] and EPAM (Elementary Perceiver and Memorizer) [19]. EPAM source code is available in the Feigenbaum archives at Stanford, and archival work to locate complete source code for GPS is in progress. Now that a proven IPL-V interpreter exists, running these programs should require substantially less effort than the initial construction of the interpreter itself.

The interpreter, the transcribed LT code, and complete outputs as described in this paper are open sourced at <https://github.com/jeffshrager/IPL-V>.

X. POSTSCRIPT: THE 1956 PROGRAM RUNS AT LAST

Shortly after a preprint of this paper appeared on arXiv, I received an email from David Moews, a mathematician previously unknown to me. He had, it turned out, just been working on the same problem—but had attacked it from the other end of the program’s history. Rather than reimplementing IPL-V, Moews had built an interpreter, in Python, for the

original IPL-I pseudocode itself, as published in the 1956 RAND report P-868 [1] and, in nearly identical form, in Newell and Simon’s 1956 IRE paper [2]. He had the code working in February 2026 [21].

The significance of this bears emphasis. As discussed in Section II, IPL-I was never implemented on any machine: it was hand-executed by Simon’s family and students simulating the imagined IPL-I machine. Every prior execution of this program in history was performed by human beings. Moews’s run is therefore, to the best of our joint knowledge, the first time the IPL-I version of the Logic Theorist has ever been executed by a machine—some seventy years after it was written, and pushing the window of “RetroAI” reanimation back to the very first AI program ever specified.

Moews reports that the printed listing contains typos and other problems and is not runnable exactly as published; it required minor patches and bug fixes, each documented in his repository [21]. One omission he found is particularly telling. Although the 1956 code is capable of finding individual proof steps, and of determining when enough steps have been found to constitute a proof, it contains no logic whatsoever to print the steps or to assemble them into the actual proof. The human simulators, of course, needed no such logic—they could see the steps they had executed and write the proof down. Moews’s solution was to have the interpreter print each potential proof step as it is found, and to write a separate program that assembles and checks the proof from the resulting set of steps. He also built a driver that imitates the 1957 “Empirical Explorations” regime [3], running the program successively on the theorems of *Principia* Chapter 2 and allowing each proof to use all axioms and all previous Chapter 2 theorems.

The two reconstructions bracket the Logic Theorist’s history: the 1956 program is the conceptual model as first specified, in roughly 400 lines that hew closely to Newell and Simon’s hypothesized cognitive operators; the 1963 program reanimated in the body of this paper is what that model became—nearly 3,000 lines—once it had to run, end-to-end, on a literal-minded machine. That the two efforts were undertaken independently, by researchers unknown to each other, and converged on complementary working reconstructions of adjacent stages of the same program, lends each a measure of replication: agreement between them is evidence that the reconstructions reflect the historical system rather than either reconstructor’s assumptions. The implications of this juxtaposition for what executable archaeology can teach us are taken up in Section VII; Moews’s interpreter, repaired source, and proof-assembly tools are available at <https://github.com/dmoews/logic-theorist>.

A PERSONAL CODA

I studied under both Newell and Simon at Carnegie Mellon University in the 1980s, where my work focused on modeling the cognition of scientists and engineers. IPL and LT were long since history by that time; I unfortunately never discussed either with either Simon or Newell—we barely learned about these pioneering efforts in our classes. But the intellectual values they instilled—rigor about mechanism, respect for the

complexity of cognition, and the conviction that thinking could be understood computationally—shaped the rest of my career. Now, at the end of that career, I find myself studying Newell, Simon, and Shaw *as* scientist-engineers: reverse-engineering their design decisions, reconstructing their reasoning, and trying to understand how they invented AI and cognitive science. The subject of my training—the cognition of scientists and engineers—and its object have folded into one another.

ACKNOWLEDGEMENTS

The following people (listed in no particular order) contributed to this project in ways ranging from code transcription to weekly encouragement: Anthony Hay, Arthur Schwarz, Leigh Klotz, David M. Berry, Rupert Lane, Amy Majczyk, Emily Davis, Ethan Ableman, and Paul McJones. I am particularly grateful to Hay, Schwarz, Klotz, and Berry, who put up with weekly progress updates throughout the project and responded with encouragement that was always helpful and sometimes technically substantive. Rupert Lane’s gridlock software caught several transcription errors that had survived multiple rounds of human review.

I am especially grateful to David Moews for generously sharing his IPL-I reanimation, for reviewing the Postscript describing it, and for the correspondence that has enriched this paper. I also thank the Computer History Museum for preserving and making available “Simon’s J’s,” the original 1962 IPL-V J-function card deck, which proved essential during debugging, and Alexander Magoun and David Walden for editorial guidance.

Finally, I want to acknowledge Allen Newell and Herbert Simon, under whom I studied at CMU in the 1980s. The subject of that training—the cognition of scientists and engineers—has, in this project, become inseparable from its object.

Portions of the debugging effort, and portions of the drafting of this paper, were carried out with the aid of various LLMs, but all concepts and final editorial decisions were the author’s alone.

REFERENCES

- [1] A. Newell and H. A. Simon, *The Logic Theory Machine – A Complex Information Processing System*, RAND Corp., Santa Monica, CA, USA, Rep. P-868, 1956.
- [2] A. Newell and H. A. Simon, “The logic theory machine—A complex information processing system,” *IRE Trans. Inf. Theory*, vol. 2, no. 3, pp. 61–79, Sep. 1956.
- [3] A. Newell, J. C. Shaw, and H. A. Simon, “Empirical explorations with the logic theory machine: A case study in heuristics,” in *Proc. Western Joint Comput. Conf.*, 1957, pp. 218–230.
- [4] A. Newell and J. C. Shaw, “Programming the logic theory machine,” in *Proc. Western Joint Comput. Conf.*, 1957, pp. 230–240.
- [5] A. N. Whitehead and B. Russell, *Principia Mathematica*. Cambridge, UK: Cambridge Univ. Press, 1910–1913.
- [6] A. Newell, et al. *Information Processing Language-V Manual*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1964.
- [7] J. McCarthy, “Recursive functions of symbolic expressions and their computation by machine, Part I,” *Commun. ACM*, vol. 3, no. 4, pp. 184–195, Apr. 1960.
- [8] H. A. Simon, *Models of My Life*. Cambridge, MA, USA: MIT Press, 1996.
- [9] Pamela McCorduck Collection, Carnegie Mellon University Archives.
- [10] R. Storey, “B7094” [Online]. Available: <https://github.com/Bertoid1311/B7094> (Accessed: Mar. 11, 2026).
- [11] E. Stefferud, “The Logic Theory Machine: A Model Heuristic Program,” RAND Corp., Santa Monica, CA, USA, Rep. RM-3731-CC, 1963.

- [12] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, “ELIZA Reanimated: Restoring the mother of all chatbots to one of the world’s first time-sharing systems,” *IEEE Ann. Hist. Comput.*, vol. 47, no. 1, pp. 8–23, 2025.
- [13] F. Gruenberger, “The History of the JOHNNIAC,” RAND Corp., Santa Monica, CA, USA, Rep. RM-5654-PR, 1968.
- [14] P. McCorduck, *Machines Who Think*, 2nd ed. Natick, MA, USA: A. K. Peters, 2004.
- [15] A. Bochanek, “Simon’s J’s,” Computer History Museum Blog, Sep. 26, 2012. [Online]. Available: <https://computerhistory.org/blog/simons-js/> (Accessed: Mar. 11, 2026).
- [16] R. Lane, “Gridlock: Accurate code extraction from PDFs,” 2024. [Online]. Available: <https://github.com/ruptl/gridlock>
- [17] J. S. Millstein, “The Logic Theorist in LISP,” *Comput. J.*, vol. 11, no. 1, pp. 39–44, Feb. 1968.
- [18] A. Newell, J. C. Shaw, and H. A. Simon, “Report on a general problem-solving program,” in *Proc. Int. Conf. Inform. Process.*, 1959, pp. 256–264.
- [19] E. A. Feigenbaum, “The simulation of verbal learning behavior,” in *Proc. Western Joint Comput. Conf.*, 1961, pp. 121–132.
- [20] H. B. Richman, J. J. Staszewski, and H. A. Simon, “Simulation of expert memory using EPAM IV,” *Psychol. Rev.*, vol. 102, no. 2, pp. 305–330, 1995.
- [21] D. Moews, “Recreation of the 1956 IPL-I version of the Logic Theorist theorem prover,” 2026. [Online]. Available: <https://github.com/dmoews/logic-theorist> (Accessed: Jun. 9, 2026).
- [22] J. Shrager, Output from the reanimated logic theorist, [Online]. Available: https://github.com/jeffshrager/IPL-V/blob/master/major_results/20260309_CompleteAndCorrect.drb (Accessed: Mar. 11, 2026).

APPENDIX

To illustrate a complex proof generated by the reconstructed Logic Theorist, the following derivation corresponds to theorem *4.25 of *Principia Mathematica*:

$$P = (P \vee P)$$

This states the idempotence of disjunction. In the LT representation, the equivalence operator is defined using implication and conjunction:

$$(P = Q) \equiv ((P \rightarrow Q) \wedge (Q \rightarrow P))$$

The Logic Theorist proof trace proceeds as follows:

- 1) **Expand the definition of equivalence:** LT begins by reducing the goal to a conjunction of two implications:

4.25 P=(PVP)
 .0 (PI (PVP)) * ((PVP) IP) 4.01 ,0 REPLACEMENT

Logic form:

$$P = (P \vee P) \Rightarrow (P \rightarrow (P \vee P)) \wedge ((P \vee P) \rightarrow P)$$

- 2) **Identify the first implication:** The system identifies a forward-chaining path to establish the first half of the conjunction:

PI (PVP) 4.20 ,0 FORWARD CHAINING

Logic form:

$$P \rightarrow (P \vee P)$$

- 3) **Find the proof via substitution:**

LT utilizes Theorem *2.20 from its memory as the basis for substitution: *Theorem 2.20 is not given as an axiom, but is a previously proved theorem that it remembered and applied here; See comment below.*

GIVEN 2.20 AI (AVB)
 SUBSTITUTION .0 PI (PVP)

Logic form:

The general theorem $A \rightarrow (A \vee B)$ is retrieved

Substituting $A := P$ and $B := P$ yields $P \rightarrow (P \vee P)$

- 4) **Complete the equivalence:** LT completes the chain by linking the proved implications back to the original goal:

GIVEN 4.20 A=A
 FORWARD CHAINING 4.25 P=(PVP)
 Q.E.D.0

Logic form:

Because both $P \rightarrow (P \vee P)$ and $(P \vee P) \rightarrow P$ are established, the equivalence $P = (P \vee P)$ holds.

Performance Metrics:

EFFORT	LIMIT 20000	ACTUAL 8727
SUBPROBLEMS	LIMIT 50	ACTUAL 2
SUBSTITUTIONS	LIMIT 50	ACTUAL 3

The successful execution of complex derivations such as this one, which use previously proved theorems, demonstrates the Logic Theorist's capacity for a primitive form of machine learning: by treating previously proved theorems, such as 2.20, as established "givens" for subsequent search paths, the system bootstraps its own knowledge base, mirroring the cumulative nature of algebraic reasoning.